Ordered pairs, products, and relation types

Ordered pairs satisfy $(a, b) = (c, d) \iff a = c \land b = d$. Coordinates in $A \times B = \{(a, b) : a \in A, b \in B\}$ are ordered. A binary relation from A to B is a subset $R \subseteq A \times B$.

For $A = \{p, q\}, B = \{0, 1, 2\}$, there are six product points and $2^6 = 64$ relations. Finite sizes m, n generally give mn points and 2^{mn} relations. The empty product still has the one subset $\varnothing$.

Define $\operatorname{Dom} R = \{a : \exists b, (a, b) \in R\}$ and $\operatorname{Ran} R = \{b : \exists a, (a, b) \in R\}$. The inverse relation $R^{-1} = \{(b, a) : (a, b) \in R\}$ always exists but need not be a function.

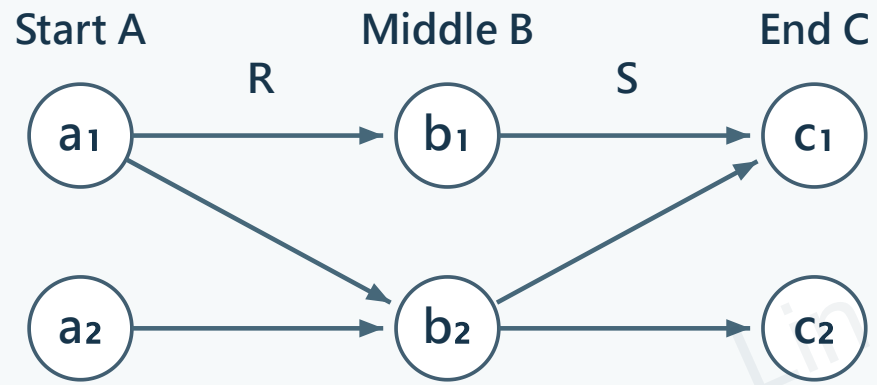
Elements $((a, b), c)$ of $(A \times B) \times C$ differ from $(a, (b, c))$ of $A \times (B \times C)$. Natural bijections do not make their ordered-pair sets literally equal.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Composing relations through witnesses

Relation composition: R first, then S

Two different intermediate witnesses yield one ordered pair



Two witnesses for (a1, c1)

a1 → b1 → c1

a1 → b2 → c1

Existence: 1 ∨ 1 = 1

Path count: 1 + 1 = 2

Arrows are ordered pairs; a pair is not repeated when it has multiple paths.

Let $A = \{a_1, a_2\}$, $B = \{b_1, b_2\}$, and $C = \{c_1, c_2\}$. Follow $R = \{(a_1, b_1), (a_1, b_2), (a_2, b_2)\}$ and then $S = \{(b_1, c_1), (b_2, c_1), (b_2, c_2)\}$. A pair (a, c) belongs to $S \circ R$ exactly when some $b \in B$ satisfies $(a, b) \in R$ and $(b, c) \in S$.

Boolean matrix product, entry by entry

Use rows a_1, a_2 and columns b_1, b_2 for M_R; use rows b_1, b_2 and columns c_1, c_2 for M_S. Boolean multiplication uses AND followed by OR:

$$M_R = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad M_S = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad M_{S \circ R} = M_R \odot M_S = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Input and output	Terms through b_1, b_2	Result
(a_1, c_1)	$(1 \wedge 1) \vee (1 \wedge 1)$	1
(a_1, c_2)	$(1 \wedge 0) \vee (1 \wedge 1)$	1
(a_2, c_1)	$(0 \wedge 1) \vee (1 \wedge 1)$	1
(a_2, c_2)	$(0 \wedge 0) \vee (1 \wedge 1)$	1

Thus $S \circ R = A \times C$. Two witnesses make the first Boolean entry 1, since it records existence; the ordinary arithmetic product gives 2 there, counting intermediate paths. With the row convention the matrices are $M_R \odot M_S$, whereas the relation is $S \circ R$; both mean R first, then S.

Function existence and uniqueness are separate

The statement $f : A \to B$ requires exactly one output for each input:

$$\forall a \in A \; \exists b \in B : (a, b) \in f,$$

$$\forall a, b, c : \; ((a, b), (a, c) \in f) \Rightarrow b = c.$$

The relation $\{(p, 0), (q, 0)\}$ is a function. Omitting q violates existence; assigning both zero and one to p violates uniqueness. Distinct inputs may share outputs.

The domain is A, codomain B, and image $f(A)$. Codomain is part of the type. Squaring from reals to reals and from reals to nonnegative reals has different surjectivity.

For every B, one empty function $\varnothing \to B$ exists. No nonempty domain maps to the empty set. Partial functions allow undefined inputs and are not total functions on the original domain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Images and preimages need no inverse function

For $X \subseteq A, Y \subseteq B$,

$$f(X) = \{f(x) : x \in X\}, \qquad f^{-1}(Y) = \{a \in A : f(a) \in Y\}.$$

Preimages exist even when f is not invertible. Integer squaring has preimages $f^{-1}(\{4\}) = \{-2, 2\}$ and $f^{-1}(\{-1\}) = \varnothing$.

Elementwise, $a \in f^{-1}(Y \cap Z) \iff f(a) \in Y \wedge f(a) \in Z$, proving preservation of intersections. Replacing conjunction with disjunction or negation proves preservation of unions and complements relative to the stated domain and codomain.

Images satisfy only $f(X \cap W) \subseteq f(X) \cap f(W)$. Under squaring, $X = \{-2\}, W = \{2\}$ give empty left side and right side $\{4\}$. If f is injective, the two preimages for a shared image coincide, proving the reverse inclusion and equality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preimages of the floor function

Floor function: preimages are sets of inputs

Preimage of {0}: 0 included, 1 excluded



Preimage of {−1,0,1}: −1 included, 2 excluded



Preimage of (0,1): no integer outputs

Filled endpoint: included Open endpoint: excluded

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Let $g : \mathbb{R} \to \mathbb{R}$, $g(x) = \lfloor x \rfloor$. From $k \le x < k+1$, $g^{-1}(\{0\}) = [0,1)$ and $g^{-1}(\{-1,0,1\}) = [-1,2)$. Every output is an integer, so $g^{-1}((0,1)) = \varnothing$. Here $g^{-1}(S)$ denotes a set preimage. Since g is neither injective nor surjective, it has no inverse function $\mathbb{R} \to \mathbb{R}$.

Injectivity, surjectivity, and their negations

Injectivity requires $\forall a, a' \in A, f(a) = f(a') \Rightarrow a = a'$. Its negation is one collision between distinct inputs. Surjectivity requires $\forall b \in B \; \exists a \in A : f(a) = b$; its negation is an output missed by every input. A bijection has both properties.

For $f : \mathbb{Z} \to \mathbb{Z}, f(n) = 3n - 7$, cancellation proves injectivity. The image is $2 + 3\mathbb{Z}$, so it is not onto integers. Restricting the codomain to that image makes it bijective, with inverse $m \mapsto (m + 7)/3$.

For finite sets, injection gives $|A| \leq |B|$ and surjection the reverse inequality. If both sizes are n, an injection's n different outputs fill the codomain. A surjection's n nonempty fibers can each contain only one of the n inputs. Thus injection, surjection, and bijection are equivalent in this equal-finite-size case, including zero.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibers separate a partition from its labels

For $b \in f(A)$, let $F_b = f^{-1}(\{b\})$. Fibers are nonempty, disjoint, and cover A, because every input has exactly one output.

With $\pi(a) = F_{f(a)}$, $\bar{f}(F_b) = b$, and inclusion $\iota : f(A) \hookrightarrow B$,

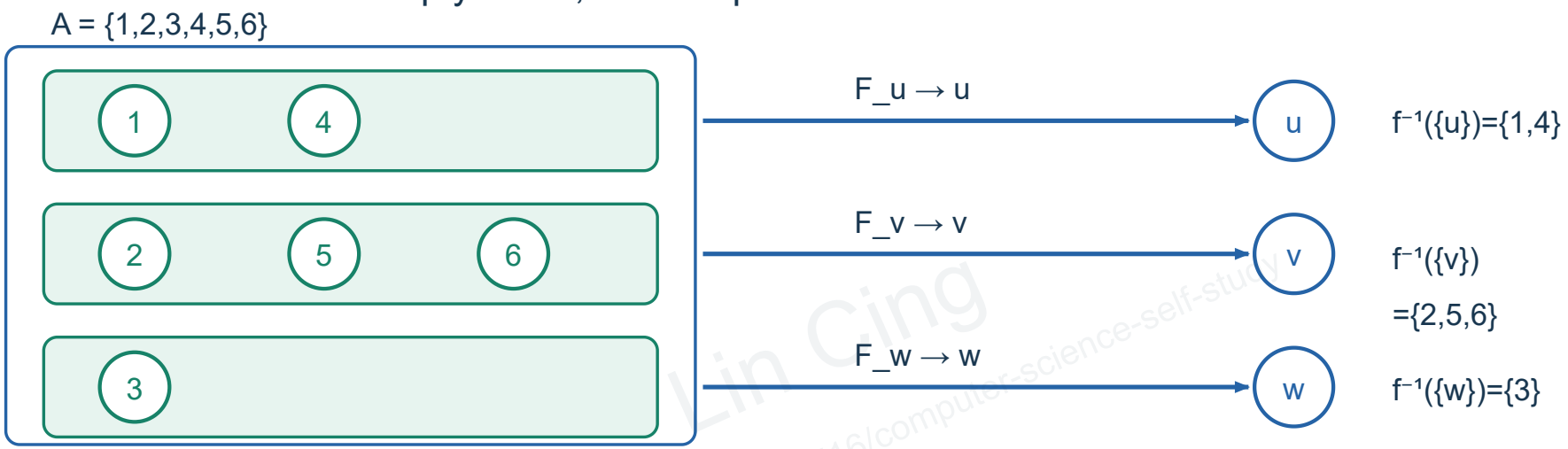
$$f = \iota \circ \bar{f} \circ \pi.$$

The middle map bijects blocks with the image. Injectivity means all fibers have at most one point; surjectivity means every codomain point has a nonempty fiber.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Example: fibers of six inputs

One function: three nonempty fibers, then output labels



The fibers $\{1, 4\}, \{2, 5, 6\}, \{3\}$ carry labels u, v, w. First send an input to its block, then its block to the output.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Equivalence relations and partitions determine each other

Define $a \sim a' \iff f(a) = f(a')$. Equality proves reflexivity, symmetry, and transitivity, with fibers as equivalence classes.

For any equivalence relation, each a lies in $[a]$. If $[a] \cap [b]$ contains c, then $a \sim c \sim b$, giving $a \sim b$. Any $x \sim a$ then satisfies $x \sim b$, and conversely, so the classes agree. Classes therefore partition the set.

Conversely, define equivalence by membership in the same block of a partition. Unique block membership proves reflexivity and symmetry. If a, b share a block and b, c share one, their blocks intersect at b and thus coincide, proving transitivity. These constructions are inverse. For the empty set, its empty partition gives the same conclusion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quotient formulas must be independent of representatives

To define $h : A/\sim \to C$ by $h([a]) = g(a)$, the necessary and sufficient condition is $a \sim a' \Rightarrow g(a) = g(a')$. Necessity follows because equal classes are one input. Sufficiency follows because any representative gives the same unique output.

The map $[a]_6 \mapsto [a]_3$ is well-defined, since divisibility by six implies divisibility by three. The proposed $[a]_4 \mapsto [a]_3$ fails: $[0]_4 = [4]_4$, but their outputs $[0]_3, [1]_3$ differ. Substitution on every integer does not alone define a function on classes.

This is well-definedness, logically before injectivity. Different representatives of the same input are not two different colliding inputs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting functions, injections, and partial functions

For finite $|A| = m$, $|B| = n$, functions number n^m. Ordered distinct output choices give

$$(n)_m = n(n-1)\cdots(n-m+1) = \frac{n!}{(n-m)!} \quad (m \le n)$$

injections, zero if $m > n$. Equal-size bijections number $n!$. The empty domain gives one function, using combinatorial convention $0^0 = 1$; nonempty-to-empty gives zero.

For partial functions with arbitrary defined subdomain, each input has n output choices or an undefined choice, giving $(n+1)^m$. Of these, n^m are total. Requiring nonempty defined subdomain excludes the one wholly undefined function.

Relations number 2^{mn} by independent cell membership. Functions instead choose exactly one cell per input row, so arbitrary subsets overcount them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stirling numbers and boundary cases

The second-kind Stirling number $S(m, k)$ counts partitions of m labeled elements into k unlabeled nonempty blocks. Set $S(0, 0) = 1$, and set $S(m, 0) = 0$ for $m > 0$, $S(0, k) = 0$ for $k > 0$, and $S(m, k) = 0$ for $k > m$.

Classify by the last element. As a singleton, deleting it leaves $S(m - 1, k - 1)$ choices. Otherwise partition the others into k blocks and choose one to join, giving $kS(m - 1, k)$. These reversible, disjoint, exhaustive constructions prove

$$S(m, k) = S(m - 1, k - 1) + kS(m - 1, k) \quad (m, k \geq 1).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stirling recurrence: a numerical table

$m \backslash k$	1	2	3	4
1	1	0	0	0
2	1	1	0	0
3	1	3	1	0
4	1	7	6	1
5	1	15	25	10
6	1	31	90	65

For example $S(6,3) = 15 + 3 \cdot 25 = 90$. Showing four columns does not limit the possible block count to four.

Two formulas for surjections

A surjection has exactly n nonempty fibers. Partition the domain, then biject its blocks to named outputs:

$$\operatorname{Surj}(m, n) = n! S(m, n).$$

Alternatively weight each function by alternating subsets of its missed outputs. If it misses t values, its net weight is $\sum_{j=0}^{t} (-1)^j \binom{t}{j} = (1-1)^t$, one exactly when $t = 0$. Exchange finite sums to get

$$\operatorname{Surj}(m, n) = \sum_{j=0}^{n} (-1)^j \binom{n}{j} (n-j)^m.$$

This proves the formula directly rather than treating later inclusion-exclusion as a black box. For $m < n$, no surjection exists and the sum is zero; for equal positive sizes it is $n!$. The empty-to-empty case has one empty function.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Six items and three named destinations

Send six labeled items to u, v, w, using every destination. Partition then label to obtain $3!S(6,3) = 6 \cdot 90 = 540$. Unnamed nonempty groups would number only ninety.

Check by omitted destinations: all $3^6 = 729$, subtract three choices of an empty destination each leaving $2^6 = 64$, and restore three constant functions missing two destinations. Missing all three is impossible for a nonempty domain. Thus $729 - 192 + 3 = 540$.

With independent uniform choices, the probability is $540/729 = 20/27$. Unequal probabilities preserve the integer count but invalidate direct division by 729. The displayed fibers $\{1,4\}, \{2,5,6\}, \{3\}$ represent one such function; swapping output names changes it.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Classifying functions by image size

Choose the k-point image, then map onto it. Image sizes give disjoint exhaustive classes:

$$n^m = \sum_{k=0}^{\min(m,n)} \binom{n}{k} k! S(m,k) = \sum_k (n)_k S(m,k).$$

For $m = 4, n = 3$, image sizes one, two, three contribute 3, $\binom{3}{2} \cdot 2! \cdot 7 = 42$, $3! \cdot 6 = 36$, totaling 81.

First-kind numbers $c(m,k)$ instead count permutations with k cycles. In the recurrence below $m, k \geq 1$ are integers; set $c(0,0) = 1$, and use 0 for out-of-range indices and the remaining zero-size cases. A new element forms a singleton cycle or is inserted after one of the $m-1$ old elements, giving

$$c(m,k) = c(m-1,k-1) + (m-1)c(m-1,k).$$

The constructions are reversible, with the same zero-size and out-of-range boundaries as S. For example $c(3,1) = 2$, unlike $S(3,1) = 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Closure before unary and binary operations

A unary operation is $u : A \to A$; here a binary operation follows the closed convention $* : A \times A \to A$. The reference also permits terminology $A \times A \to B$, checking closure separately; repeated operations here explicitly require it.

Addition is closed on $\mathbb{N}_0$; subtraction is not, since $2 - 5 \notin \mathbb{N}_0$, though it is closed on integers. Division is not total on $\mathbb{R}^2$ because zero denominators are undefined.

Union and intersection are closed binary operations on $\mathcal{P}(U)$, and complement is unary relative to fixed U. A sequence is a function $\mathbb{N}_0 \to A$, whose input is its index rather than the previous value.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Commutativity, associativity, and identities

Commutativity requires $a * b = b * a$, associativity $(a * b) * c = a * (b * c)$, for all inputs. A symmetric operation table establishes only commutativity, not the three-input associativity condition.

Real averaging $a * b = (a + b)/2$ commutes but is not associative: $(0 * 0) * 4 = 2$, whereas $0 * (0 * 4) = 1$. Integer subtraction is closed but neither commutative nor associative.

A two-sided identity satisfies $e * a = a = a * e$. If e, e' are both identities, $e = e * e' = e'$, proving uniqueness without associativity. Left identities alone need not be unique: for $a * b = b$, every element is a left identity, and a set with more than one element has no right identity.

The minimum operation on an ordered set has its maximum as identity when one exists; on all integers it has none. The domain matters as much as the operation's name.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Free cells in operation tables

Let $|A| = n \geq 1$. A closed binary operation assigns one of n values to each of n^2 input pairs, giving n^{n^2} tables.

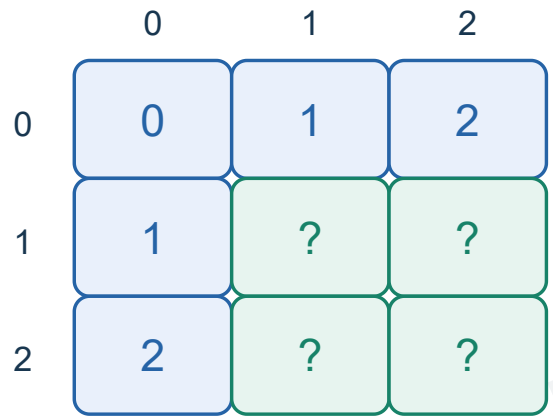
For commutativity, the n diagonal and $n(n-1)/2$ upper off-diagonal cells are free; symmetry fixes the lower half. Thus $n(n+1)/2$ cells are chosen independently, giving $n^{n(n+1)/2}$ tables.

A specified two-sided identity e fixes its row and column, each with n cells; their intersection (e, e) was counted twice, so $2n - 1$ cells are fixed. The remaining $n^2 - (2n-1) = (n-1)^2$ cells give $n^{(n-1)^2}$ tables. A two-sided identity is unique, so specified-identity classes are disjoint; requiring some identity gives $n \cdot n^{(n-1)^2}$ tables.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Example: an operation table with a fixed identity

Identity 0 fixes row 0 and column 0



Fixed: 0*x = x = x*0

Free: four cells, three values each

$3^4 = 81$ operations

May be nonassociative; need not be groups.

For $A = \{0, 1, 2\}$ with identity zero, four cells give 81 operations. Requiring commutativity leaves three independent cells and gives 27. Associativity has not been imposed, so such tables need not define groups.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floor, ceiling, and negative boundaries

The definitions $\lfloor x \rfloor = k \iff k \le x < k+1$ and $\lceil x \rceil = k \iff k-1 < x \le k$ imply $\lceil x \rceil = -\lfloor -x \rfloor$ and $\lfloor x + t \rfloor = \lfloor x \rfloor + t$ for integer t.

Write $x = p + \alpha, y = q + \beta$, with integer p, q and fractional parts in $[0, 1)$. Then

$$\lfloor x + y \rfloor = \lfloor x \rfloor + \lfloor y \rfloor + \lfloor \alpha + \beta \rfloor.$$

The last term is zero or one, so floor is not additive. With $x = -1.4, y = 0.7$, the left side is minus one while the separate floors sum to minus two. Truncation toward zero also differs from floor on negative inputs.

The convention $0! = 1$ counts an empty permutation. Real $\log_b x$ requires $x > 0, b > 0, b \neq 1$; positive-cost comparisons commonly also require $b > 1$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Generalized pigeonhole bounds are tight

Place $N \geq 0$ objects into $k \geq 1$ boxes. If every box had fewer than $\lceil N/k \rceil$, the total would be at most $k(\lceil N/k \rceil - 1) < N$, contradiction. Some box has at least that many. The zero-object case holds; zero boxes cannot serve as a divisor.

Write $N = qk + r, 0 \leq r < k$. Put $q + 1$ in r boxes and q in the others, attaining the bound. Thus it cannot generally be improved by one. To force at least $t \geq 1$ together, the minimum total is $k(t - 1) + 1$.

The impossibility of injections from larger finite sets to smaller ones is the one-per-box version. Boxes must cover all objects and their number must be correctly bounded. The theorem asserts existence without identifying the box.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pigeonhole principle: a 21-game consecutive block

Prefix-sum collision: a consecutive block of 21 games

First list: $a_1 < \cdots < a_{77} = 132$

Second list: $a_1+21 < \cdots < a_{77}+21 = 153$

154 entries lie in 1...153; each list is strictly increasing, so a collision crosses lists.

$a_j = a_i + 21 \Rightarrow i < j \Rightarrow$ days $i+1$ through j contain 21 games

The collision determines i and j; no particular schedule is assumed.

Over 77 days, 132 games are played, with at least one every day. Let a_i be the cumulative games through day i. Both 77-entry lists lie in $\{1, \ldots, 153\}$ and are strictly increasing, so their 154 entries force a cross-list collision $a_j = a_i + 21$. Strict increase implies $i < j$, and days $i + 1$ through j contain exactly $a_j - a_i = 21$ games.

Prefix residues produce a contiguous interval

Given an integer $m \geq 1$ and integers $a_1, \ldots, a_m$, let $s_0 = 0$ and $s_j = \sum_{t=1}^{j} a_t$. Including the empty prefix gives $m + 1$ prefixes, each with a residue modulo m, but only m possible residues.

The pigeonhole principle gives distinct indices, which we order as $0 \leq i < j \leq m$, with $s_i \equiv s_j \pmod{m}$. Subtracting cancels the first i terms:

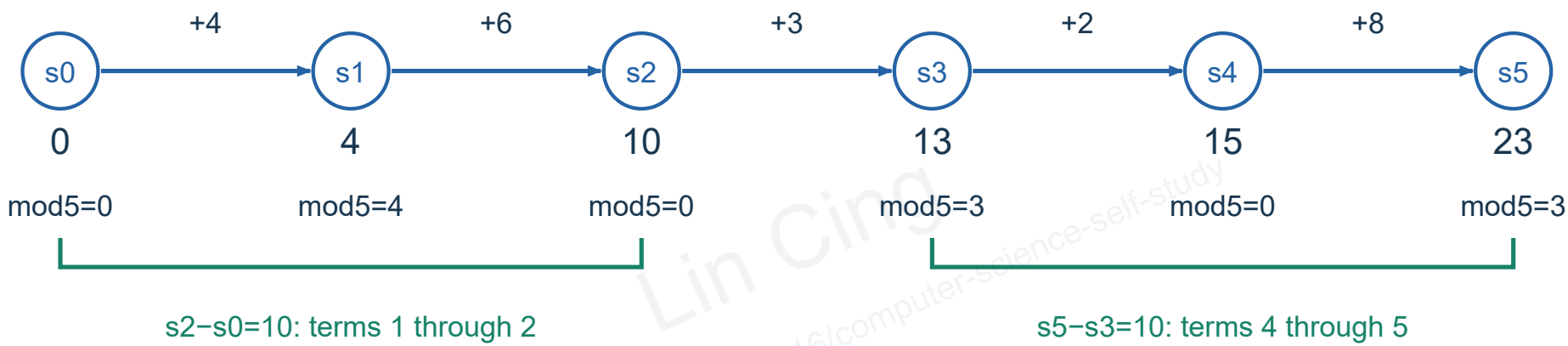
$$s_j - s_i = a_{i+1} + a_{i+2} + \cdots + a_j \equiv 0 \pmod{m}.$$

Because $i < j$, the contiguous segment contains at least one term, and its sum is divisible by m.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Example: prefix residues and a contiguous interval

Prefix indices and congruence: subtract prefixes for the intervening interval



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For $4, 6, 3, 2, 8$, prefixes are $0, 4, 10, 13, 15, 23$, with residues $0, 4, 0, 3, 0, 3$ modulo five. Positions zero and two give $4 + 6 = 10$; three and five give $2 + 8 = 10$. The interval begins at $i + 1$, not i. Prefix zero includes intervals starting at the first item.

Odd-part boxes force divisibility

Let $n \geq 1$ be an integer. Every positive integer uniquely has form $2^t u$ with odd u. Among $n+1$ different choices from $\{1, \ldots, 2n\}$, classify by the n possible odd parts $1, 3, \ldots, 2n-1$. Two choices $2^r u, 2^s u$ share a box. Their distinctness gives $r \neq s$, so the lower power divides the higher.

The bound is tight. The n integers $n+1, \ldots, 2n$ have every larger element below twice every smaller one, excluding divisibility between distinct choices. Parity alone would not suffice: odd numbers need not divide each other. Choose classes whose shared membership forces the desired relation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A two-label proof for monotone subsequences

Let r, s be positive integers and consider a finite sequence of distinct reals. Let I_j, D_j be longest strictly increasing and decreasing subsequence lengths ending at position j. If $i < j, a_i < a_j$, appending gives $I_j > I_i$; if $a_i > a_j$, it gives $D_j > D_i$. Thus every position's label pair differs.

If no increasing subsequence has length r and no decreasing one length s, all labels lie in $[r-1] \times [s-1]$, allowing at most $(r-1)(s-1)$ positions. A longer sequence must contain one of them.

Ten distinct reals therefore contain a length-four monotone subsequence. Nine do not guarantee it: $3, 2, 1, 6, 5, 4, 9, 8, 7$ has longest lengths three in both directions. Subsequences retain order without needing contiguity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Composition types, associativity, and iteration

For $f : A \to B, g : B \to C$, define $(g \circ f)(a) = g(f(a))$. More generally $f(A) \subseteq \operatorname{Dom} g$ suffices, but specify the final domain and codomain.

Compatible triple compositions evaluate as $h(g(f(a)))$ with either parenthesization, proving associativity. Identity maps satisfy $f \circ \mathrm{id}_A = f = \mathrm{id}_B \circ f$. Composition need not commute: on integers, $f(x) = x + 2, g(x) = 3x$ give $gf(x) = 3x + 6$, $fg(x) = 3x + 2$.

For a self-map, define $f^0 = \mathrm{id}_A$, $f^{t+1} = f \circ f^t$. This exponent counts compositions, not numerical powers of values. The example gives $f^t(x) = x + 2t$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The exact condition for injective composition

For $f : A \to B$ and $g : B \to C$, the composite $g \circ f$ is injective iff f is injective and g is injective on $f(A)$.

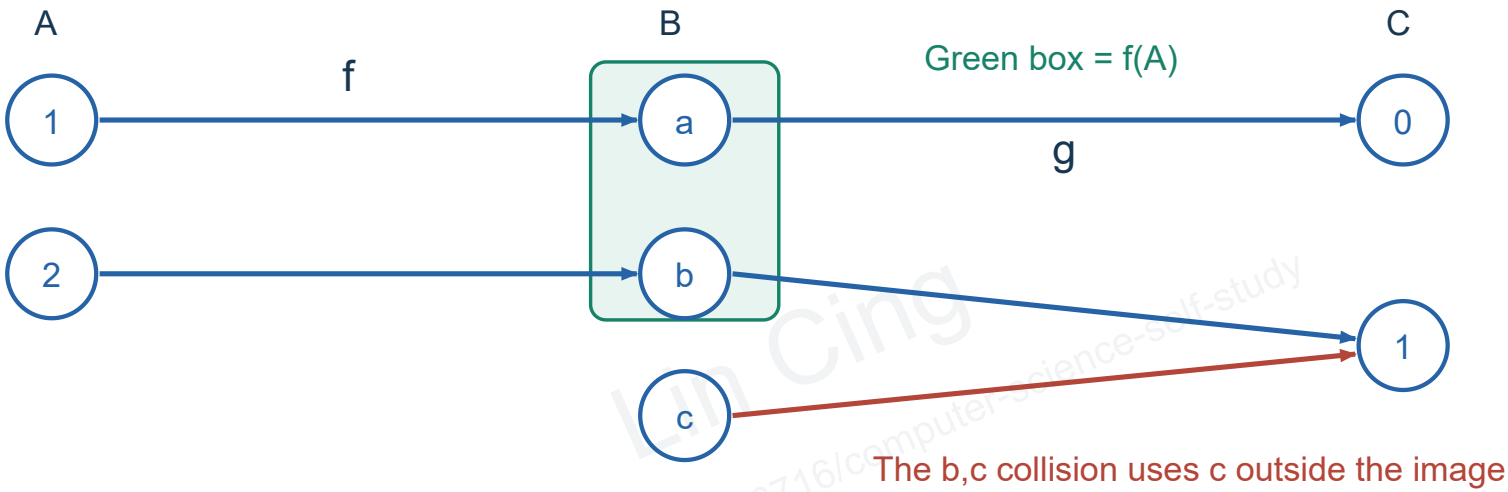
Forward, $f(a) = f(a')$ implies $g(f(a)) = g(f(a'))$; composite injectivity gives $a = a'$, so f is injective. For any $y, y' \in f(A)$, choose $y = f(a)$ and $y' = f(a')$. If $g(y) = g(y')$, composite injectivity gives $a = a'$ and hence $y = y'$; thus g is injective on the image.

Conversely, for any $a, a' \in A$, if $g(f(a)) = g(f(a'))$, injectivity of g on $f(A)$ gives $f(a) = f(a')$. Injectivity of f then gives $a = a'$, which is exactly injectivity of the composite.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Example: a collision outside the image

g is not injective on B, but is injective on f(A)



With $f(1) = a, f(2) = b$ and $g(a) = 0, g(b) = g(c) = 1$, g is not globally injective but is injective on $\{a, b\}$, so the composite is injective. Inferring global outer-map injectivity overlooks the restricted range being used.

The image condition for surjective composition

The exact condition is $g(f(A)) = C$. Thus surjective composition forces $g : B \to C$ onto, but need not force $f : A \to B$ onto. Take $A = \{1\}, B = \{a, b\}, C = \{0\}$, with $f(1) = a, g(a) = g(b) = 0$.

If g is additionally injective, each $b \in B$ has $g(f(a)) = g(b)$ for some a by composite surjectivity. Cancelling g proves $f(a) = b$, hence onto.

Two surjections compose to a surjection by finding a preimage under g first, then under f. Two injections compose by cancelling the outer map first, then the inner. Preimage search runs opposite to function evaluation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Invertibility is equivalent to bijectivity

If $gf = \mathrm{id}_A$, $fg = \mathrm{id}_B$, the first proves injectivity of f, and the second supplies preimage $g(b)$ for every b, proving surjectivity.

Conversely, a bijection gives each b exactly one preimage, defining $g(b)$ and both identity equations. Any other inverse h satisfies $h = h(fg) = (hf)g = g$, so the inverse is unique. Compatible bijections obey $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$, undoing the outer map first.

Squaring on nonnegative reals is invertible with nonnegative square root. From all reals to nonnegative reals it is onto but not invertible because of sign collisions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Left inverses, right inverses, and empty sets

A left inverse $\ell : B \to A$ with $\ell f = \mathrm{id}_A$ forces injectivity. Conversely, if f is injective and $A \neq \varnothing$, take unique preimages on $f(A)$, sending unused outputs to a fixed $a_0 \in A$. The injection $\varnothing \to B \neq \varnothing$ has no left inverse because no $B \to \varnothing$ exists.

A right inverse r with $fr = \mathrm{id}_B$ forces surjectivity. For a finite surjection, choose one point in each nonempty fiber. The universal infinite version requires the axiom of choice, whereas unique preimages of a bijection require no arbitrary choices.

On $\mathbb{N}_0$, let $f(n) = n + 1, \ell(m) = \max(m - 1, 0)$. Then $\ell f = \mathrm{id}$, but $f\ell(0) = 1 \neq 0$. If both a left and right inverse exist, $\ell = \ell(fr) = (\ell f)r = r$, making them one two-sided inverse.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Infinite cardinality and finite intuition

On $\mathbb{N}_0$, doubling is injective but not onto; flooring division by two is onto but not injective. Equinumerous sets do not turn a specified injection into a surjection.

Doubling bijects natural numbers with evens. A bijection to integers is $0 \mapsto 0, 2k - 1 \mapsto k, 2k \mapsto -k$ for $k \geq 1$. Positive, negative, and zero outputs each occur once, and their signs reconstruct unique indices.

No map $F : A \to \mathcal{P}(A)$ is onto. If it were, the set $D = \{a : a \notin F(a)\}$ would equal $F(d)$ for some d, giving $d \in D \iff d \notin D$, contradiction. This general diagonal proof is not finite enumeration. Cardinality asks whether some bijection exists, not whether every existing map has the same properties.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Quantifiers and constants in growth bounds

For eventually nonnegative costs and eventually positive comparison functions,

$$f \in O(g) \iff \exists c > 0, N \, \forall n \geq N : 0 \leq f(n) \leq cg(n).$$

A lower bound $\Omega(g)$ instead requires $f(n) \geq cg(n)$, and $\Theta(g)$ requires positive lower and upper constants past one threshold. Signed-function versions use absolute values rather than negative execution times.

Constants depend on the functions but cannot vary with n. Big-O is an upper bound, not a synonym for worst case; best, average, and worst costs each admit growth bounds. Two-sided bounds imply both upper and lower bounds. Conversely, use the maximum of their thresholds to prove $\Theta(g) = O(g) \cap \Omega(g)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Proving growth with general inequalities

For $n \geq 1$, $4n^2 \leq 4n^2 + 3n + 2 \leq 9n^2$, giving $\Theta(n^2)$ witnesses 4,9,1. To prove $n^2 \notin O(n)$, any proposed fixed c, N fails at integer $n > \max(c, N)$, since $n^2 > cn$. A few large examples do not refute all constants.

Transitivity follows by $f \leq cg \leq cdh$ beyond both thresholds. Nonnegative cost bounds may similarly be added or multiplied after their thresholds.

For fixed bases $a, b > 1$, $\log_a n = \log_b n / \log_b a$, a positive constant multiple, hence the same $\Theta(\log n)$ class. A varying base such as $\log_n n = 1$ cannot be ignored.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fix input size and cost model first

An array's input size may be its element count n. A positive integer N has binary length $b = \lfloor \log_2 N \rfloor + 1$; cost linear in N may be exponential in b.

A fixed-word model may count indexing, comparison, and addition at unit cost. Arbitrary-precision arithmetic and string comparisons require bit or character costs. An operation count is not automatically a time measurement under every representation.

For the legal inputs of a fixed size, define worst cost as the supremum, possibly $+\infty$, and best cost as the infimum; these are a maximum or minimum only when attained. For example, an array of one arbitrarily long string can require unbounded character comparisons; include total character length in the size to distinguish these inputs. Average cost requires a distribution $p(x)$, with $E[T] = \sum p(x)T(x)$; the sum may be infinite. "Usually" alone supplies no distribution. Correctness also needs an input contract, output specification, invariant, and termination proof. Before comparing speed, establish that the procedures solve the same problem.

Linear search and its full correctness contract

For $A[0..n-1]$ and target x, return its first equal index, or n if absent. Short-circuit evaluation checks the index before reading the array.

```
i=0
while i<n and A[i]!=x:
    i=i+1
return i
```

At each loop start, $0 \le i \le n$ and every earlier element differs from x. This holds initially on an empty prefix. Entering the body establishes $A[i] \neq x$, so incrementing preserves it. Nonnegative $n - i$ decreases each iteration, proving termination.

At exit with $i < n$, equality holds and the invariant proves it is the first match. At $i = n$, the invariant proves absence. At $n = 0$, return zero without any array access.

Best, worst, and average linear-search costs

Count element comparisons. For $n \geq 1$, a first match at position i, index $i-1$, costs i; absence costs n. Best is one and worst n. If absence probability is $0 \leq q < 1$ and successful positions are conditionally uniform,

$$E[C] = (1-q)\frac{n+1}{2} + qn.$$

When $q = 1$, conditioning on success is undefined because success has probability zero; every search fails, so directly $E[C] = n$, agreeing with the numerical value of the displayed expression at $q = 1$. For $n = 4, q = 1/3$, each successful position has probability $1/6$, contributing $1/6, 2/6, 3/6, 4/6$, and absence contributes $4/3$, totaling three. With all successful mass at the first cell, the average can instead be one when every search succeeds.

For arbitrary position probabilities $p_1, \ldots, p_n$ and absence probability q summing to one, the formula is $\sum ip_i + nq$. Sorting, index construction, and variable-length comparison costs are not included.

Nested-loop ranges determine the cost

```
for i=1..n:
    for j=1..i: op()
```

The count $\sum_{i=1}^{n} i = n(n+1)/2$ lies between $n^2/2$ and n^2 for $n \geq 1$, hence $\Theta(n^2)$. Empty input gives zero operations, though program control still has constant cost.

If each inner loop instead starts $j = 1$, calls `op()` while $j \leq n$, and doubles j, it takes $\lfloor \log_2 n \rfloor + 1$ iterations per outer iteration, giving $\Theta(n \log n)$ for $n \geq 2$. A fixed c-iteration inner loop gives $cn = \Theta(n)$. Two nested loops do not by themselves determine a quadratic power. Also prevent index overflow and retain the stated operation model.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Binary exponentiation and its invariant

For a in a set with associative multiplication and identity one, and integer $N \geq 0$, define $a^0 = 1$.

```
r=1; b=a; e=N
while e>0:
    if e is odd: r=r*b
    b=b*b
    e=floor(e/2)
return r
```

The invariant is $rb^e = a^N$. At even $e = 2t$, the new product is $r(b^2)^t = rb^{2t}$. At odd $e = 2t + 1$, it is $(rb)(b^2)^t = rb^{2t+1}$. Associativity suffices because factors are powers of the same b, without general commutativity. Initialization satisfies the invariant; at $e = 0$, it gives the answer.

For positive e, flooring its half strictly decreases it, proving termination. At $N = 0$, the loop is skipped and returns one. The final unused square may be omitted as an optimization, while preserving initialization and multiplication rules.

Thirteen's bits and the actual iteration states

The bits of $13 = (1101)_2$ read low first are 1,0,1,1. To compute 3^{13}:

Initial e	b	Initial r	Multiply into r?	Final r
13	3	1	Yes, low bit 1	3
6	9	3	No, low bit 0	3
3	81	3	Yes, low bit 1	243
1	6561	243	Yes, low bit 1	1594323

https://github.com/xinhi6716/computer-science-self-study

Multiplication count and bit cost

Four squarings and three extra multiplications total seven. Generally $N \geq 1$ gives $b = \lfloor \log_2 N \rfloor + 1$ iterations, and popcount extra multiplications, at most $2b$ multiplications in total.

This is not a logarithmic bit-time proof for unbounded integers. Answer length can grow with N. Modular computation with positive modulus keeps intermediate values bounded, but large-modulus multiplication and reduction still need their own cost analysis.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 5.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study